

Performance of a processor

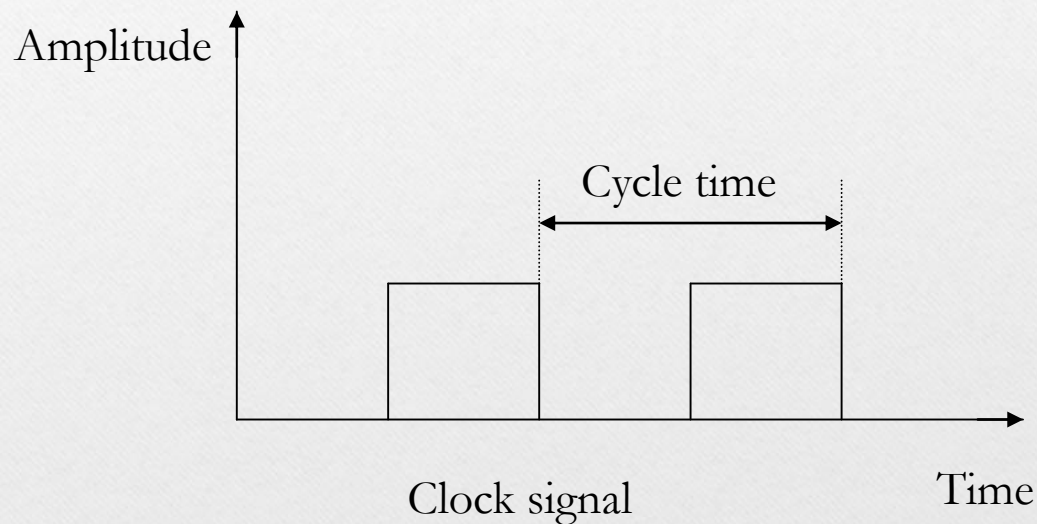
ANISHA M. LAL

Performance of a Processor

Parameters

- Instruction Execution Rate - average Cycles Per Instruction CPI for a program.
- Processor Time
- MIPS – Million Instruction Per Second
- MFLOPS – Million Floating point operations
- Benchmarks programs
- Speedup-Amdahls law

Performance Measures



Frequency – Number of time periods or clock cycle per second.

Cycle count – Cycles in a particular time interval.

Cycle Time – Time period ($1/f$)

Instruction Execution Rate

- Instruction execution rate is calculated by the average cycles per instruction CPI for a program
- If all instructions required the same number of clock cycles, then CPI would be a constant value for a processor.
- However, on any give processor, the number of clock cycles required varies for different types of instructions, such as load, store, branch, and so on.

Instruction Execution Rate

$$CPI = \frac{\sum_{i=1}^n (CPI_i \times I_i)}{I_c}$$

In the above formula

- i – Instruction type
- CPI_i - The number of cycles required for that type of instruction i
- I_i – The number of executed instructions of type i for a given program.
- I_c - instruction count of a program which has the number of machine instructions executed for that program until it runs to completion or for some defined time interval

Calculate CPI for five different instruction types given below.

- Load (5 cycles)
- Store (4 cycles)
- R-type (4 cycles)
- Branch (3 cycles)
- Jump (3 cycles)
- If a program has:
 - 50% load instructions
 - 15% R-type instructions
 - 25% store instructions
 - 8% branch instructions
 - 2% jump instructions

$$\text{CPI} = \frac{5 \times 50 + 4 \times 25 + 4 \times 15 + 3 \times 8 + 3 \times 2}{100} = 4.4$$

Processor Time

- The processor time T needed to execute a given program can be expressed as

$$T = I_c \times CPI \times \tau$$

- A processor is driven by a clock with a **constant frequency f** or, equivalently, a constant cycle time τ , where $\tau = 1/f$.
- During the execution of an Instruction, part of the work is done by the processor, and part of the time a word is being transferred to or from memory.
- In this latter case, the time to transfer depends on the memory cycle time, which may be greater than the processor cycle time.
- We can rewrite the preceding equation as,
$$T = I_c \times [p + (m \times k)] \times \tau$$
- Where,
- p – the number of processor cycles needed to decode and execute the instruction,
- m – the number of memory references needed
- k – The ratio between memory cycle time and processor cycle time

MIPS

- **Instructions per second (IPS)** is a measure of a computer's processor speed.
- A common measure of performance for a processor is the rate at which instructions are executed, expressed as millions of instructions per second (MIPS), referred to as the **MIPS rate**.
- We can express the MIPS rate in terms of the clock rate and CPI as follows:

$$\text{MIPS rate} = \frac{I_c}{T \times 10^6} = \frac{f}{CPI \times 10^6}$$

MIPS Example

- Consider the execution of a program which results in the execution of 2 million instructions on a 400-MHz processor.
- The instruction mix and the CPI for each instruction type are given below based on the result of a program trace experiment:

Instruction Type	CPI	Instruction Mix
Arithmetic and logic	1	60%
Load/store with cache hit	2	18%
Branch	4	12%
Memory reference with cache miss	8	10%

$$CPI = 0.6 + (2 \times 0.18) + (4 \times 0.12) + (8 \times 0.1) = 2.24.$$

$$\text{MIPS rate is } (400 \times 10^6) / (2.24 \times 10^6) \approx 178.$$

- A 400 MHz processor was used to execute a benchmark program with the following instruction mix and clock cycle count:

Instruction TYPE	Instruction count	Clock cycle count
Integer Arithmetic	45000	1
Data transfer	32000	2
Floating point	15000	2
Control transfer	8000	2

Determine the effective CPI, MIPS (Millions of instructions per second) rate, and execution time for this program.

$$\text{CPI} = \frac{45000 \times 1 + 32000 \times 2 + 15000 \times 2 + 8000 \times 2}{100000} = \frac{155000}{100000} = 1.55$$

$$\text{Effective processor performance} = \text{MIPS} = \frac{\text{clock frequency}}{\text{CPI}} \times \frac{1}{1 \text{ Million}} = \frac{400,000,000}{1.55 \times 1000000} = \frac{400}{1.55} = 258 \text{ MIPS}$$

$$\text{Execution time}(T) = \text{CPI} \times \text{Instruction count} \times \text{clock time}$$

$$\frac{\text{CPI} \times \text{Instruction Count}}{\text{frequency}} = \frac{1.55 \times 100000}{400 \times 1000000} = \frac{1.55}{4000} = 0.0003875 \text{ sec} = 0.3875 \text{ ms}$$

MFLOPS

- Floating point performance is expressed as millions of floating-point operations per second (MFLOPS), defined as follows:

$$\text{MFLOPS rate} = \frac{\text{Number of executed floating-point operations in a program}}{\text{Execution time} \times 10^6}$$

Benchmarks

- Measures such as MIPS and MFLOPS have proven inadequate to evaluating the performance of processors.
- Because of differences in instruction sets, the instruction execution rate is not a valid means of comparing the performance of different architectures.
- For example, consider this high-level language statement:

$$A = B + C$$

- With the traditional instruction set architecture, referred to as a complex instruction set computer (CISC), this instruction can be compiled into one processor instruction:

add mem(B), mem(C), mem (A)

Benchmarks

- On a typical RISC machine, the compilation would look something like this:

load mem(B), reg(1);

load mem(C), reg(2);

add reg(1), reg(2), reg(3);

store reg(3), mem (A)

- Because of the nature of the RISC architecture, both machines may execute the original high-level language instruction in about the same time.
- If this example is representative of the two machines, then if the CISC machine is rated at 1 MIPS, the RISC machine would be rated at 4 MIPS.
- But both do the same amount of high-level language work in the same amount of time.

Benchmarks

- Measure the performance of systems using a set of benchmark programs.
- The same set of programs can be run on different machines and the execution times compared.
- A benchmark suite is a collection of programs, defined in a high-level language, that together attempt to provide a representative test of a computer in a particular application or system programming area.
- Collection of benchmark suites is defined and maintained by the System Performance Evaluation Corporation (SPEC), an industry consortium.
- Ex: SPEC CPU2006, SPECjvm98, SPECweb99

Speed up

- Measure of how much faster the computation executes versus the best serial code
-

$$\text{Speed up} = \frac{\text{Serial time}}{\text{Parallel time}}$$

- Example: *Painting a picket fence*
 - 30 minutes of preparation (serial)
 - One minute to paint a single picket
 - 30 minutes of cleanup (serial)
- Thus, 300 pickets takes 360 minutes (

Computing Speedup

Number of painters	Time	Speedup
1	$30 + 300 + 30 = 360$	1.0X
2	$30 + 150 + 30 = 210$	1.7X
10	$30 + 30 + 30 = 90$	4.0X
100	$30 + 3 + 30 = 63$	5.7X
Infinite	$30 + 0 + 30 = 60$	6.0X

Speedup

- The **speedup** is defined as the ratio of the serial runtime of the best sequential algorithm for solving a problem to the time taken by the parallel algorithm to solve the same problem on p **processors**.
- The efficiency is defined as the ratio of **speedup** to the number of **processors**.

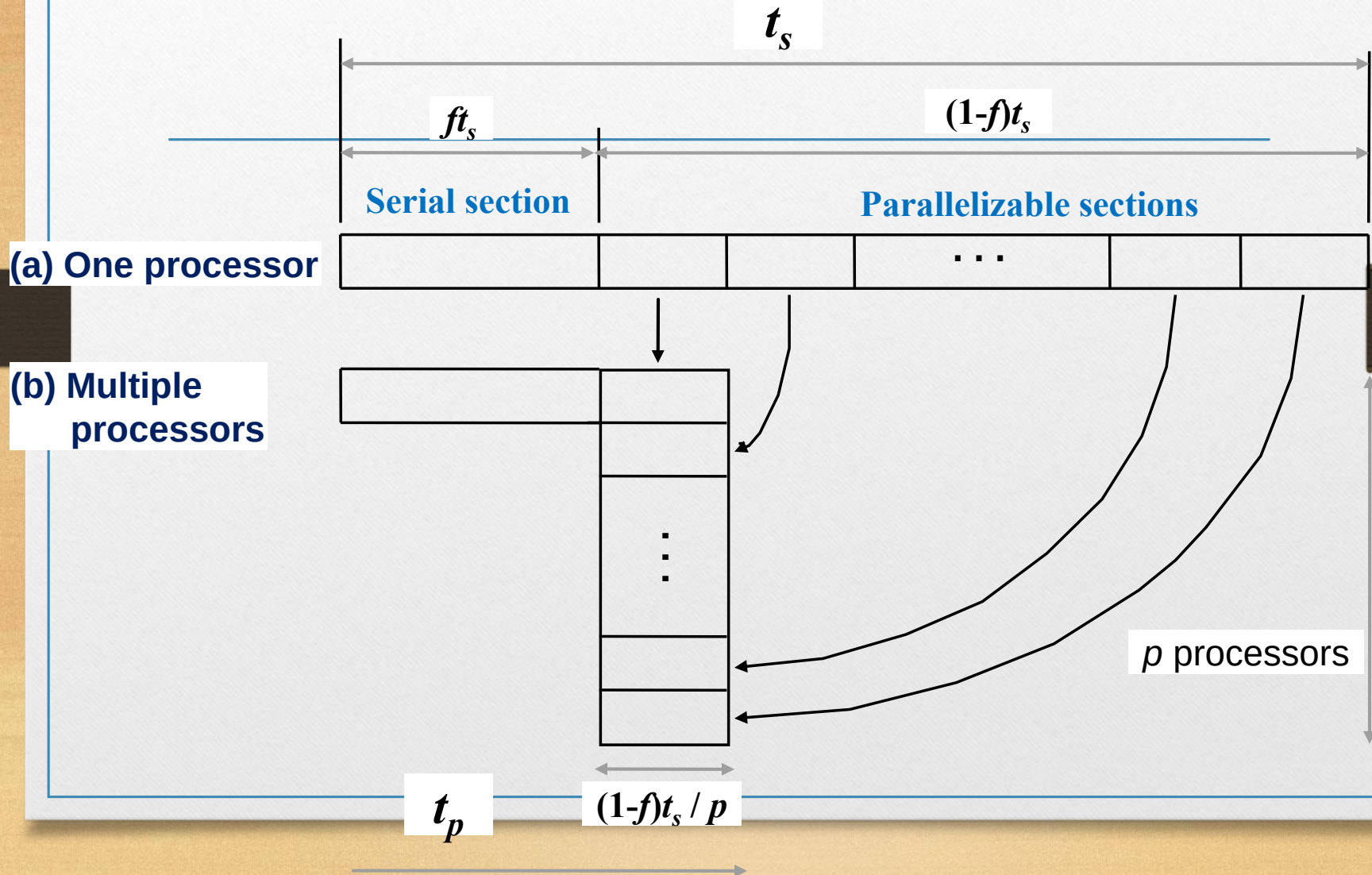
Amdahl's law

- Amdahl's law is a model for the expected speedup and the relationship between parallelized implementations of an algorithm and its sequential implementations, under the assumption that the problem size remains the same when parallelized.
- In the case of parallelization, Amdahl's law states that if $(1-f)$ is the proportion of a program that can be made parallel (i.e., benefit from parallelization), and f is the proportion that cannot be parallelized (remains serial), then the maximum speedup that can be achieved by using P processors is

$$\text{Speedup} = \frac{1}{f + \frac{(1-f)}{P}}$$

Maximum Speedup

Amdahl's law



Speedup factor is given by:

$$S(p) = \frac{t_s}{ft_s + \frac{(1-f)t_s}{p}} = \frac{1}{f + \frac{(1-f)}{p}} \quad \lim_{p \rightarrow \infty} S(p) = \frac{1}{f}$$

This equation is known as **Amdahl's law**.

Even with infinite number of processors, maximum speedup is limited to $1/f$.

Problems

- If 90% of the computation can be parallelized, what is the max. speedup achievable using 8 processors?

$$\begin{aligned}\text{Speedup} &= \frac{1}{f + \frac{(1-f)}{P}} \\ &= \frac{1}{0.1 + (1-0.1)/8} = 4.7\end{aligned}$$

References

- **Text Book** – William Stallings, “Computer Organization and Architecture, Designing for performance”, 8th edition, Prentice Hall.
- Internet Sources.